

# ATELIER C++



```
#include <iostream>

int main() {
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```



# Plan du cours

- Introduction générale
  - Programmation Procédurale
  - Programmation Orienté Objet POO
- Langage C++ : Notion de base
- Éléments de langage

# PROGRAMMATION PROCEDURALE (1)

- La **programmation procédurale** est un **paradigme de programmation** basé sur l'organisation du code en **procédures ou fonctions**.
- Une **procédure** (ou fonction) est un ensemble d'instructions qui effectue une tâche spécifique.
- Les programmes procéduraux sont généralement **linéaires** et **structurés**.

Identifier le  
problème

Décomposer le  
problème en  
sous problèmes

Définir les  
fonctions  
correspondantes

Ecrire le  
programme  
principal en  
pseudo code

Implémenter  
Exécuter  
Débugger

# PROGRAMMATION PROCEDURALE (2)

- **Caractéristiques principales**

- 1. Organisation en fonctions :**

- Chaque tâche est isolée dans une fonction.
- Exemple : une fonction pour additionner deux nombres, une autre pour afficher un message.

- 2. Séquence d'instructions :**

- Le programme suit un flux séquentiel : exécution des instructions **ligne par ligne**.

- 3. Données globales ou locales :**

- Les **variables** peuvent être globales (accessibles partout) ou locales (accessibles seulement dans une fonction).

- 4. Contrôle de flux :**

- Utilisation de **conditions** (if, switch) et **boucles** (for, while) pour contrôler l'exécution du programme.

- 5. Réutilisation limitée :**

- Les fonctions permettent de réutiliser du code, mais la gestion de grandes quantités de données devient complexe.

# PROGRAMMATION PROCEDURALE (3)

## Avantages



- ✓ Facile à apprendre
- ✓ Réutilisation et maintenabilité claire
- ✓ Approche séquentielle claire
- ✓ Organisation et clarté du code

## Limites



- × Difficulté à gérer des programmes complexes ou très grands
- × Difficile de représenter des objets du monde réel (comme un employé, une voiture, etc.)
- × Moins flexible pour la **réutilisation de code** comparé à la programmation orientée objet



# Transition vers la Programmation Orientée Objet (POO)

## Pourquoi passer à la POO ?

- La **programmation procédurale** fonctionne bien pour des programmes simples.
- Mais quand un programme devient **grand et complexe**, il devient difficile de gérer :
  - Les **données** dispersées
  - Les **fonctions dépendantes**
  - La **réutilisation du code**
- **La solution** : regrouper les données et les fonctions qui les manipulent dans une même entité, appelée objet.

# PROGRAMMATION ORIENTE OBJET POO (1)

- La **Programmation Orientée Objet (POO)** est un **paradigme de programmation** basé sur l'organisation du code autour des objets, qui représentent des entités du monde réel.
- **Objet** : instance d'une classe, contenant **données (attributs)** et **comportements (méthodes)**.
- **Classe** : modèle ou plan de construction pour créer des objets.
- **Exemple conceptuel** :

| Objet réel | Attributs(données)       | Méthodes                           |
|------------|--------------------------|------------------------------------|
| Voiture    | couleur, marque, vitesse | Démarrer(), freiner(), klaxonner() |
| Employé    | nom, âge , salaire       | Travailler(), sePrésenter()        |

# PROGRAMMATION ORIENTE OBJET POO (2)

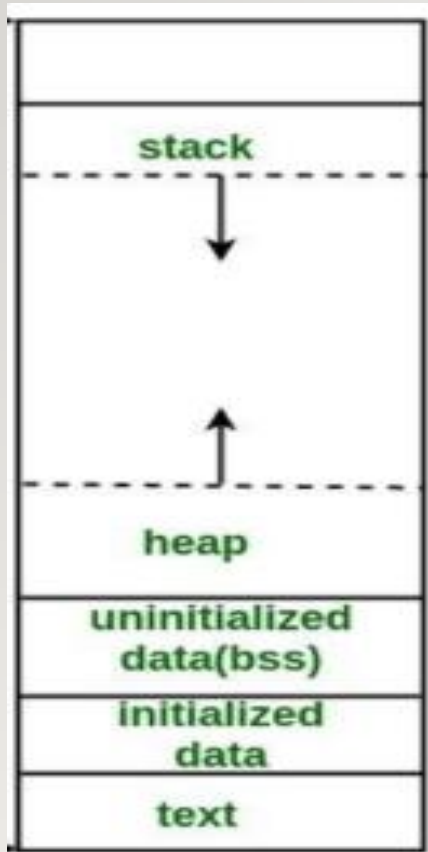
- Les points forts du POO :
  - **Modularité** : les objets forment des modules compacts regroupant des données et un ensemble d'opérations.
  - **Abstraction** : Les entités objets de la POO sont proches de celles du monde réel. Les concepts utilisés sont donc proches des abstractions familières que nous exploitons.
  - **Productivité et réutilisabilité** : Plus l'application est complexe et plus l'approche POO est intéressante en termes de productivité.
  - **Sûreté** : L'encapsulation et le typage des classes offrent une certaine robustesse aux applications.



# Qu'est-ce que le C++ ?

- Langage compilé, dérivé du C mais avec la programmation orientée objet.
- Un programme écrit en langage C++ ressemble beaucoup à un programme écrit en langage C, à la différence près qu'il contient essentiellement des classes.
- Utilisé en : systèmes embarqués, jeux vidéo, applications hautes performances, etc.
- Un programme C++ est constitué au minimum d'une fonction main().
- Le langage C++ possède assez peu d'instructions, il fait par contre appel à des bibliothèques
- **Exemples:**
  - math.h : bibliothèque de fonctions mathématiques
  - iostream.h : bibliothèque d'entrées/sorties standard
  - complex.h : bibliothèque contenant la classe des nombres complexes.

# FORME DE LA MÉMOIRE EN PROGRAMME C++



- **Code (ou Text Segment)** : Contient le **code compilé** du programme (les instructions machine). C'est une zone **en lecture seule** → on ne peut pas la modifier pendant l'exécution.

Exemple : la fonction `main()` ou `cout << "Hello";`.

- **Données statiques (Data Segment)** : Stocke les **variables globales, statiques** et les **constantes**. Il existe sous-parties :
  - **Initialized Data** → variables globales ou statiques avec une valeur initiale.
  - **Uninitialized Data (BSS)** → variables globales ou statiques **non initialisées** (remplies par défaut avec 0).
- **Pile (Stack)** : allocation temporaire des objets (existent seulement durant l'exécution d'une fonction).
- **Tas (Heap)** : allocation dynamique des objets (le programmeur alloue et désalloue la mémoire, opérateur `delete` et `free`)

# STRUCTURE D'UN PROGRAMME EN C++

```
main.cpp x
1  #include <iostream>
2  #include "fcts.h"
3  using std::cout;
4  using std::cin;
5  using std::endl;
6
7  int main()
8  {
9      int a,b;
10     cout << "Entrer un entier" << endl;
11     cin >> a;
12     cout << "Entrer un entier" << endl;
13     cin >> b;
14
15     cout << "La somme est " << somme(a,b) << endl;
16     return 0;
17 }
```

Directive préprocesseur. Utiliser une bibliothèque **standard** iostream

Utilisation d'une bibliothèque personnelle

Utilisation des objets prédéfinis dans le nom de domaine **std**

Fonction principale du programme

**cout** : la sortie standard du programme (console)

**cin** : l'entrée standard des données saisies au clavier

**endl** : fin de la ligne

**<<** : opérateur de sortie du flux de données

**>>** : opérateur d'entrée du flux de données

# CONCEPT DE BASE : INITILISATION

3 manières d'initialiser une variable :

---

( *expression-list* )      (1)

---

= *expression*      (2)

---

{ *initializer-list* }      (3)

---

```
int b = 1;  
int c(2);  
int d{}; // d vaut 0
```

```
std::cout << "b= " << b << std::endl;  
std::cout << "c= " << c << std::endl;  
std::cout << "d= " << d << std::endl;
```

Console c

```
b= 1  
c= 2  
d= 0
```



# ALLOCATION STATIQUE – ALLOCATION DYNAMIQUE

## Allocation statique

```
int x =0 ;  
int y(5);
```

Les variables sont allouées de manière permanente.

L'allocation est faite avant l'exécution du programme.

Pas de possibilité de réutilisation de la mémoire.

## Allocation dynamique

```
int *pt = new int(10);
```

Les variables ne sont allouées que si votre unité de programme est active.

L'allocation est faite pendant l'exécution du programme.

La mémoire est réutilisable et peut être libérée lorsqu'elle n'est pas nécessaire (opérateur delete).

**En c/c++, la gestion de la mémoire est à la charge du programmeur. Il doit libérer de la mémoire à chaque fois qu'elle n'est plus pointée.**



# CONCEPT DE BASE : LES BOOLEENS

```
#include <iostream>

using namespace std;

int main()
{
    bool v_bool1 = true;
    bool v_bool2(false);

    cout << v_bool1 << endl;
    cout << v_bool2 << endl;
    cout << boolalpha << v_bool1 << endl;
    cout << boolalpha << v_bool2 << endl;
    return 0;
}
```

D:\Git\dev-isimg\AtelierCPP\fiche3\bin\Debug\fiche3.exe

1  
0  
true  
false

Déclaration et initialisation

Une variable de type **bool** ne peut avoir que deux états  
**true** (1) ou **false** (0)

# CONCEPT DE BASE : LES STRINGS (1)

```
#include <iostream>

using namespace std;

int main()
{
    string lang("C plus plus");
    string niv = "CPI2";
    cout << niv << " " << lang << endl;
    // ajouter à la fin
    cout << niv.append(lang) << endl;
    // on peut utiliser aussi size() pour avoir la longueur
    cout << "La chaine contient " << niv.length() << " caracteres" << endl;
    return 0;
}
```

D:\Git\dev-isimg\AtelierCPP\fiche3\bin\Debug\fiche3.exe  
CPI2 C plus plus  
CPI2C plus plus  
La chaine contient 15 caracteres

## Opérateurs et fonctions :

**+** : concaténation

**length() / size()** : longueur

**== / !=** : comparaison

**compare(str)** : comparer

**at(index)** : lire un caractère

**insert(index,str)** : insérer

Plus de fonctions via ce lien:

<https://devdocs.io/cpp/>

**Remarque:** on peut parcourir un objet de type string avec un indice comme un tableau de caractères.

## CONCEPT DE BASE : LES STRINGS (2)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string prenom = "Amani";
    string nom = "Fallah";

    string identite = prenom + " " + nom; // concaténation avec +

    cout << identite << endl; // affiche : Amani Fallah
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string phrase = "Bonjour";
    phrase.append(" tout le monde!");

    cout << phrase << endl; // affiche : Bonjour tout le monde!
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string a = "chat";
    string b = "chien";

    cout << a.compare(b) << endl; // renvoie > 0 car "chat" > "chien"
    cout << b.compare(a) << endl; // renvoie < 0 car "chien" < "chat"
    cout << a.compare("chat") << endl; // renvoie 0 car elles sont égales
}
```

## CONCEPT DE BASE : LES STRINGS (3)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string mot = "programmation";

    cout << mot.at(0) << endl; // p (premier caractère)
    cout << mot.at(3) << endl; // g (4ème caractère)
    cout << mot.at(mot.size()-1) << endl; // n (dernier caractère)
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string mot = "programmation";

    mot.insert(6, "C++ ");
    cout << mot << endl; // affiche "prograC++ mmation"
}
```

# CONCEPT DE BASE : LES STRINGS (4)

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string lang;
```

```
    cout << "C'est quoi votre langage prefere?" << endl;
```

```
    cin >> lang;
```

```
    cout << "Vous aimez programmer avec " << lang << endl;
```

```
    return 0;
```

```
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\Debug\fiche3.exe

C'est quoi votre langage prefere?  
C plus plus  
Vous aimez programmer avec C

**cin >>** : arrête la lecture au premier caractère espace

**getline(cin, str):** arrête la lecture au premier retour à la ligne

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string lang;
```

```
    cout << "C'est quoi votre langage prefere?" << endl;
```

```
    getline(cin, lang);
```

```
    cout << "Vous aimez programmer avec " << lang << endl;
```

```
    return 0;
```

```
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\Debug\fiche3.exe

C'est quoi votre langage prefere?  
C plus plus  
Vous aimez programmer avec C plus plus



# LES CONSTANTES

En C++ on distingue deux grands types de **constantes** selon le moment où leur valeur est fixée

**Compile-time constant** (constante connue à la compilation)

```
#include <iostream>
using namespace std;

int main() {
    const int TAILLE = 10;      // constante de compilation
    constexpr double PI = 3.1415; // constante de compilation aussi

    int tab[TAILLE]; // possible car TAILLE est connu à la compilation

    cout << "PI = " << PI << endl;
}
```

**Runtime constant** (constante connue à l'exécution)

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Entrez une valeur : ";
    cin >> n;

    const int valeur = n; // runtime constant

    cout << "Vous avez saisi : " << valeur << endl;
}
```

**Remarque** : on peut utiliser constexpr seulement avec compile-time constant  
const = la valeur ne change pas une fois affectée, mais elle peut être connue seulement à l'exécution.  
constexpr = la valeur doit être connue à la compilation

# LES TABLEAUX : RAW ARRAYS

```
constexpr int SIZE = 5;
int t[SIZE];

// initialiser t
for (int i = 0; i < SIZE; i++)
    t[i] = i;

// afficher t
for (int i = 0; i < SIZE; i++)
    std::cout << t[i] << " ";
```

**Stack Memory Allocation**

```
constexpr int SIZE = 5;
int* tab = new int[SIZE];

// initialiser tab
for (int i = 0; i < SIZE; i++)
    tab[i] = i;

// afficher tab
for (int i = 0; i < SIZE; i++)
    std::cout << tab[i] << " ";

delete[] tab;
```

**Heap Memory Allocation**

# LES TABLEAUX : STATIC ARRAYS

```
#include <iostream>
#include <array>
#include <algorithm>

int main()
{
    std::array<int, 5> tab{33,5,8,-1, 88};

    // trier tab avec la fonction prédéfinie sort
    std::sort(tab.begin(), tab.end());

    // afficher tab
    for (int i = 0; i < tab.size(); i++)
        std::cout << tab[i] << " ";

    return 0;
}
```

Autre façon de parcourir

```
for (const auto& ele : tab)
    std::cout << ele << " ";
```

**Stack Memory Allocation**

# LES CONTENEURS

En C++, un **conteneur** est une classe de la bibliothèque standard (STL – *Standard Template Library*) qui sert à **stocker et organiser des collections d'objets**.

➤ Exemples de conteneurs :

`std::vector` (tableau dynamique)

`std::list` (liste chaînée)

`std::map` (dictionnaire clé/valeur)

`std::set` (ensemble sans doublons)

# LES CONTENEURS - VECTOR

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;           // vecteur vide d'entiers
    v.push_back(10);        // ajoute 10 à la fin
    v.push_back(20);
    v.push_back(30);

    cout << "Taille = " << v.size() << endl; // 3
    cout << "v[1] = " << v[1] << endl;      // 20

    v.pop_back();           // enlève le dernier élément (30)
}
```

- ✓ Le vector est le conteneur le plus proche d'un **raw array dynamique**, mais avec beaucoup plus de fonctionnalités.
- ✓ Un **vector** peut **grandir et rétrécir automatiquement** avec :
  - push\_back() → ajoute à la fin
  - pop\_back() → enlève le dernier élément
  - resize() → change la taille
- ✓ Quelques fonctions utiles de vector :
  - v.size() → nombre d'éléments
  - v.empty() → vrai si le vecteur est vide
  - v.clear() → supprime tous les éléments
  - v.front() → premier élément
  - v.back() → dernier élément
  - v.at(i) → élément avec vérification de la borne (plus sûr que v[i])
  - v.insert(pos, val) → insère un élément à une position donnée
  - v.erase(pos) → supprime un élément à une position



# LES CONTENEURS - VECTOR

```
#include <iostream>
#include<vector>
using namespace std;
int main()
{
    vector<int> vl;
    // remplissage de vl
    for (int i = 1; i <= 5; i++)
        vl.push_back(i);
    // affichage avec un itérateur
    cout << "affichage dans l'ordre" << endl;
    for (auto it = vl.begin(); it != vl.end(); ++it)
        cout << *it << " ";
    cout << endl;
    cout << "affichage dans l'ordre inverse" << endl;
    for (auto it = vl.rbegin(); it != vl.rend(); ++it)
        cout << *it << " ";
    return 0;
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\De

affichage dans l'ordre

1 2 3 4 5

affichage dans l'ordre inverse

5 4 3 2 1

Inclure la bibliothèque vector.

Voir :

<https://cplusplus.com/reference/vector/vector/>

Création statique d'un tableau d'entiers

Insérer des éléments dans le tableau

Début du tableau partant de la gauche

Fin du tableau partant de la gauche

Début et fin du tableau partant de la droite.

# LES CONTENEURS – MAP / UNORDRED MAP

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> ages;
    ages["Ali"] = 25;
    ages["Sara"] = 30;
    ages["Omar"] = 22;

    cout << "Age de Sara = " << ages["Sara"] << endl; // 30
}
```

- C'est comme un dictionnaire :
- Chaque **clé** est associée à une **valeur**.
- Les clés sont **uniques** et toujours **triées** automatiquement.

# LES CONTENEURS – EXERCICES

Etant donné un tableau d'entiers « tab » contenant les N (saisie au clavier) premiers entiers partant de 0. on veut l'éclater en deux autres tableaux de manière à avoir un tableau d'entiers premiers et un tableau pour les entiers non premiers.

- Ecrire la fonction « estPremier » qui retourne un booléen indiquant si l'entier est premier ou non.
- Ecrire une fonction « remplir » qui lit la valeur maximale N et remplit le tableau passé en paramètre.
- Ecrire une fonction « afficher » qui affiche les éléments d'un tableau.
- Ecrire la fonction principale main qui crée et remplit le tableau « tab » et l'éclater en deux autres tableaux telle que demander.

# LES CONTENEURS – EXERCICES

```
#include <iostream>
#include <cmath>
#include <vector>
bool estPremier(int& n) {
    bool prem = true;
    for (int i = 2; i <= sqrt(n); i++)
        if (n % i == 0)
            prem = false;
    return prem;
}
```

```
void remplir(std::vector<int>& v) {
    int n;
    std::cout << "Donner la limite des entiers:\n";
    std::cin >> n;
    for (int i = 0; i < n; i++)
        v.push_back(i);
}
```

```
void afficher(std::vector<int>& vec) {
    for (int& el : vec)
        std::cout << "[" << el << "]";
    std::cout << std::endl;
}
```



# LES CONTENEURS – EXERCICES

```
int main()
{
    std::vector<int> tab{};
    remplir(tab);
    std::vector<int> vecPremier{}, vecNonPremier{};
    for (int& ele : tab)
        if (estPremier(ele) == true)
            vecPremier.push_back(ele);
        else
            vecNonPremier.push_back(ele);
    std::cout << "Les nombres premiers:\n";
    afficher(vecPremier);
    std::cout << "Les nombres non premiers:\n";
    afficher(vecNonPremier);
    return 0;
}
```



# UNE VARIABLE PLUSIEURS TYPE

Depuis c++17, il est possible d'avoir une seule variable capable d'avoir plusieurs types.

```
#include <iostream>
#include <variant>

int main()
{
    std::variant<std::string, int, float> data;

    data = std::string("hello");
    std::cout << std::get<std::string>(data) << std::endl;

    data = 120;
    std::cout << std::get<int>(data) << std::endl;

    data = 1.03f;
    std::cout << std::get<float>(data) << std::endl;
    return 0;
}
```

**std::variant<>**

Accès aux membres du variant

# UNE VARIABLE PLUSIEURS TYPE - EXERCICE

Soit **f** une fonction définie par  $\sqrt{(x-1) * (2-x)}$

La fonction retourne une valeur réelle lorsqu'elle est définie, sinon elle retourne un message d'erreur « la fonction n'est pas définie !! ».

Ecrire une fonction principale qui appelle la fonction **f** avec une valeur saisie au clavier.

```
#include <iostream>
#include <variant>
std::variant<double, std::string> f(double& x) {
    double r = (x - 1) * (2 - x);

    if (r > 0)
        return sqrt(r);
    else
        return "Erreur sur le signe de l'expression !!";
}
```

# UNE VARIABLE PLUSIEURS TYPE - EXERCICE

```
int main()
{
    double val;
    std::cout << "Donner une valeur:";
    std::cin >> val;
    auto resultat = f(val);
    if (resultat.index() == 0)
        std::cout << "f(" << val << ")=" << std::get<0>(resultat);
    else
        std::cout << std::get<1>(resultat);
    return 0;
}
```

Vérifier si le premier objet est actif

Accéder (lire) le contenu du variant

# STD::TUPLE / STD::PAIR

En C++, `std::pair` et `std::tuple` servent à regrouper plusieurs valeurs dans une seule variable, mais avec des différences

- Un `std::pair` est utilisé pour combiner **exactement** deux objets qui peuvent être de types différents.

- `#include <utility>`

- On utilise `first` et `second` pour accéder aux objets contenus dans le `pair`

- Un `std::tuple` est un objet qui peut contenir plusieurs objets. Les objets peuvent être de différents types.

- Les objets contenus dans un `tuple` sont construits dans leur ordre d'accès.

- On accède aux éléments avec `std::get<index>(tuple)`

- `#include <tuple>`

```
#include <iostream>
#include <utility> // pour std::pair
using namespace std;

int main() {
    pair<int, string> p = {1, "Bonjour"};
    cout << p.first << " - " << p.second << endl;
}
```

```
#include <iostream>
#include <tuple>
using namespace std;

int main() {
    tuple<int, string, double> t = {42, "Salut", 3.14};

    cout << get<0>(t) << ", " << get<1>(t) << ", " << get<2>(t) << endl;
}
```

# STRUCTURED BINDINGS / TUPLE / PAIR

```
#include <iostream>
#include <tuple>

std::tuple<std::string, std::string, int> creerMoi()
{
    return("Amani", "FALLAH", 2025)
}

int main()
{
    auto [prenom, nom, annee] = creerMoi();
    cout << prenom << " " << nom << " " << annee << endl;

    return 0;
}
```

Espace mémoire structuré  
(en général  
utilisé une seule fois, pour  
éviter de  
créer un objet `Personne`  
par exemple)



# STRUCTURED BINDINGS / TUPLE / PAIR

## Exercice

Ecrire une fonction `décomposer` qui prend en paramètre une chaîne de caractère et retourne le jour, le mois et l'année séparément.

Ecrire une fonction principale `main` qui saisie la date sous forme `mm/jj/aaaa` et appelle la fonction `décomposer`, puis affiche le résultat.

On peut utiliser `std::stoi` pour convertir un `string` en un `int`

# STRUCTURED BINDINGS / TUPLE / PAIR

## Solution

```
#include <iostream>
#include <string>
#include <tuple>

std::tuple<int, int, int> decomposer(std::string& date) {
    int day = std::stoi(date.substr(0, 2));
    int month = std::stoi(date.substr(3, 5));
    int year = std::stoi(date.substr(6, 9));

    return { day, month, year };
}
```

```
int main()
{
    std::string maDate;
    std::cout << "Donner une date sous forme aa/jj/aaaa";
    std::cin >> maDate;
    auto [jour, mois, annee] = decomposer(maDate);
    std::cout << "Jour:" << jour << std::endl;
    std::cout << "Mois:" << mois << std::endl;
    std::cout << "Annee:" << annee << std::endl;
}
```

# LVALUE et RVALUE

En général, on les utilise selon l'expression suivante:

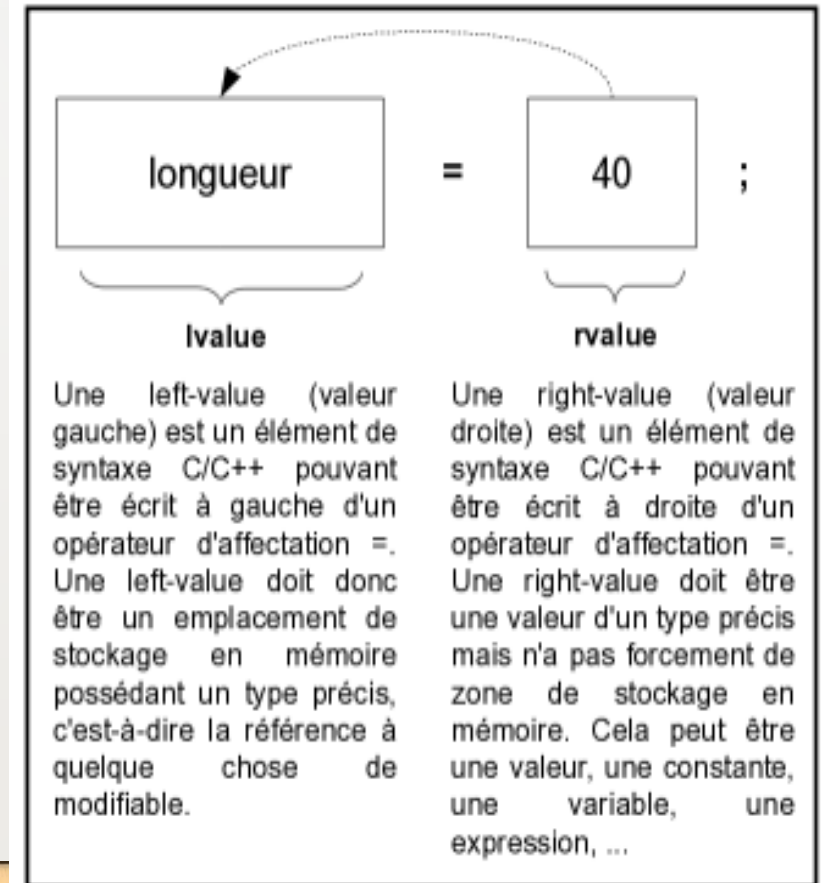
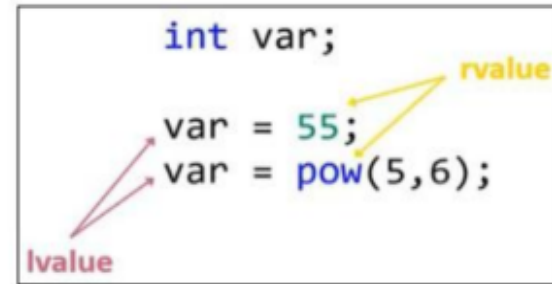
$\text{lvalue} = \text{rvalue}$

**Lvalue** est un objet qui a une location identifiée en mémoire (a une adresse en mémoire).

- Capable de stocker une information
- Ne peut pas être une fonction, une constante ou une expression

**Rvalue** est un objet n'ayant pas un identifiant en mémoire (désigne la valeur dans un espace mémoire).

- Toute chose pouvant retourner une expression constante ou une valeur.



# LES REFERENCES EN C++

En C++, il est possible de déclarer une référence **j** sur une variable **i** : cela permet de créer **un nouveau nom j qui devient synonyme de i** (un alias). On pourra donc modifier le contenu de la variable en utilisant une référence. La déclaration d'une référence se fait en précisant le type de l'objet référencé, puis le symbole &, et le nom de la variable référence qu'on crée.

```
#include <iostream>

int main (int argc, char **argv)
{
    int i = 10; // i est un entier valant 10
    int &j = i; // j est une référence sur un entier, cet entier est i.
    //int &k = 44; // ligne7 : illégal

    std::cout << "i = " << i << std::endl; // i = 10
    std::cout << "j = " << j << std::endl; // j = 10

    // A partir d'ici j est synonyme de i, ainsi :
    j = j + 1; // est équivalent à i = i + 1 !

    std::cout << "i = " << i << std::endl; // i = 11
    std::cout << "j = " << j << std::endl; // j = 11

    return 0;
}
```

## Remarque:

Une référence ne peut être initialisée qu'une seule fois : lors de sa déclaration. Une référence ne peut donc référencer qu'une seule variable tout au long de sa durée de vie.



# LES REFERENCES EN C++

## → Intérêt des références :

- Comme une référence établit un lien entre deux noms, leur utilisation est efficace dans le cas de variable de grosse taille car cela évitera toute copie.
- Les références sont (systématiquement) utilisées dans le passage des paramètres d'une fonction (ou d'une méthode) dès que le coût d'une recopie par valeur est trop important ("gros" objet).

## Remarque :

Une référence peut être déclarée constante ce qui empêchera toute modification par une fonction (ou une méthode).



# TEMPLATES – FONCTIONS TEMPLATES

```
template<typename T>
T maxi(T a, T b){
    return a>b?a:b;
}

int main()
{
    float res3 = maxi(5.3,2.3);
    int res4 = maxi(10,12);
    cout << "float maxi: " << res3 << endl;
    cout << "int maxi: " << res4 << endl;
    return 0;
}
```

On peut définir plusieurs  
*parameters templates* pour  
une fonction

```
1 template <class T, class U>
2 T GetMin (T a, U b) {
3     return (a<b?a:b);
4 }
```

**Les modèles de fonction** sont des fonctions spéciales qui peuvent fonctionner avec des types génériques.

**Un paramètre de modèle** est un type spécial de paramètre qui peut être utilisé pour passer un type comme argument

On peut écrire aussi:

```
template<class T>
T maxi(T a, T b){
    return a>b?a:b;
}
```

# POINTEURS SUR LES FONCTIONS

Un pointeur de fonction contient l'adresse du début du code binaire constituant la fonction.

```
#include <iostream>
void afficherInt(int a){
    std::cout << "c'est un " << a << std::endl;
}
int main()
{
    void(*aff)(int); // pointeur sur la fonction
    aff = afficherInt;
    aff(6);
    return 0;
}
```

Le nom de la fonction représente son adresse de début.

Nom de la fonction

type (\*identificateur)(paramètres);

Les paramètre de la fonction

Type renvoyé par la fonction

# POINTEURS SUR LES FONCTIONS - EXERCICE

Etant donné un tableau d'entiers `tab`, on se propose de faire différentes manipulations sur ce tableau:

- Ecrire une fonction « `affichePair` » qui affiche un entier s'il est pair.
- Ecrire une fonction « `afficheImpair` » qui affiche un entier s'il est impair.
- Ecrire une fonction « `pourChaque` » qui applique une fonction (passée en paramètre) sur les éléments des tableaux.
- Ecrire une fonction principale `main` qui crée un tableau d'entiers et affiche pour chaque élément pair du tableau puis chaque élément impair du tableau.

# POINTEURS SUR LES FONCTIONS - CORRECTION

```
#include <iostream>
#include <vector>
void afficherPair(int a){
    if(!(a%2))
        std::cout << a << " est pair" << std::endl;
}
void afficherInPair(int a){
    if(a%2)
        std::cout << a << " est impair" << std::endl;
}
void PourChaque(const std::vector<int>& vec, void(*aff)(int)){
    for(int val : vec)
        aff(val);
}
int main()
{
    std::vector<int> tab = {3,9,2,5,7,4,12};
    PourChaque(tab, afficherPair);
    std::cin.get();
    PourChaque(tab, afficherInPair);
    return 0;
}
```



# PASSAGE DE PARAMETRE – PASSAGE PAR VALEUR

Lorsque l'on passe une valeur en paramètre à une fonction, cette valeur est copiée dans une variable locale de la fonction correspondant à ce paramètre. Cela s'appelle un **passage par valeur**.

Exemple :

```
#include <iostream>
using namespace std;

void incrementer(int x) {
    x = x + 1;    // x est une COPIE
    cout << "Dans la fonction : x = " << x << endl;
}

int main() {
    int a = 5;
    incrementer(a); // a est passé PAR VALEUR
    cout << "Dans main : a = " << a << endl;
    return 0;
}
```

Résultat :

Dans la fonction : x = 6

Dans main : a = 5

💡 Remarque : a reste 5 car la fonction a travaillé sur une copie.



# PASSAGE DE PARAMETRE – PASSAGE PAR REFERENCE

La fonction reçoit **une référence** à la variable originale, **pas une copie**. Donc les changements dans la fonction affectent la variable d'origine.

```
void incrementer_ref(int &x) { // passage par référence
    x = x + 1;
}
```

```
int a = 5;
incrementer_ref(a);
cout << "a = " << a << endl; // a est maintenant 6
```

# PASSAGE DE PARAMETRE – PASSAGE PAR POINTEUR

Similaire à référence, mais syntaxe différente

```
#include <iostream>
using namespace std;

// Fonction qui échange deux entiers via pointeurs
void echanger(int *x, int *y) {
    int temp = *x;    // on accède à la valeur pointée
    *x = *y;
    *y = temp;
}

int main() {
    int a = 10, b = 20;

    cout << "Avant échange : a = " << a << ", b = " << b << endl;

    // On passe l'adresse des variables
    echanger(&a, &b);

    cout << "Après échange : a = " << a << ", b = " << b << endl;

    return 0;
}
```

# PASSAGE DE PARAMETRE – RESUME ET COMPARAISON

| Méthode       | Syntaxe de la fonction          | Appel de la fonction    | Effet sur la variable d'origine                                                                  | Exemple                                                             |
|---------------|---------------------------------|-------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Par valeur    | <code>void f(int x)</code>      | <code>f(a);</code>      | ✅ Une <b>copie</b> est créée → l'original <b>ne change pas</b>                                   | <pre>cpp void f(int x){ x++; } int a=5; f(a); // a=5</pre>          |
| Par référence | <code>void f(int &amp;x)</code> | <code>f(a);</code>      | ⚡ Pas de copie → l'original <b>est modifié</b>                                                   | <pre>cpp void f(int &amp;x){ x++; } int a=5; f(a); // a=6</pre>     |
| Par pointeur  | <code>void f(int *x)</code>     | <code>f(&amp;a);</code> | ⚡ L'adresse est transmise → l'original <b>est modifié</b> (il faut <code>*x</code> pour accéder) | <pre>cpp void f(int *x){ (*x)++; } int a=5; f(&amp;a); // a=6</pre> |

# Notion des classes

---

## 1. Qu'est-ce qu'une classe ?

Une **classe** est un **modèle** ou un **plan** qui décrit un type d'objet.

Elle **regroupe** :

- des **données** (appelées *attributs* ou *membres de données*),
- et des **fonctions** (appelées *méthodes* ou *fonctions membres*).

👉 En résumé, **une classe = un modèle d'objet**, tandis qu'un **objet = une instance concrète** de cette classe.



# Notion des classes

```
class Voiture {  
    public: // zone accessible de l'extérieur  
        string marque;  
        int annee;  
  
        void demarrer() {  
            cout << "La voiture démarre " << endl;  
        }  
};
```

Nom de la classe

Visibilité des attributs et des méthodes

Attributs de la classe

Méthode de la classe



# Notion des classes

- Une fois la classe définie, on peut **créer un objet** (ou instance) :

```
int main() {  
    Voiture v1;           // création d'un objet de type Voiture  
    v1.marque = "Toyota";  
    v1.annee = 2022;  
  
    cout << "Marque : " << v1.marque << endl;  
    cout << "Année : " << v1.annee << endl;  
    v1.demarrer();  
  
    return 0;  
}
```

# Définition externe d'une classe

---

- Pour une meilleure représentation d'une classe en C++, on peut définir un prototype de la classe dans un **fichier .h** et la définition de ses méthodes dans un **autre fichier .cpp**

- **Fichier d'en-tête (.h) :**

Contient la déclaration de la classe, c'est-à-dire : les noms des attributs et des méthodes, les signatures des fonctions membres (sans leur corps). 👉 Ce fichier sert d'interface entre le programme et la classe.

- **Fichier source (.cpp) :**

Contient la définition des méthodes déclarées dans le fichier .h.

Chaque méthode est précédée du nom de la classe et de l'opérateur :: (appelé opérateur de portée).

# Définition externe d'une classe

## Exemple

- Le fichier **Etudiant.h**

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant {
private:
    string nom;
    int age;

public:
    Etudiant(string n, int a);
    void afficher();
};

#endif
```

On remarque l'utilisation des directives de compilation **#ifndef**, **#define** et **#endif** pour gérer les inclusions multiples du fichier header.

Attributs de la classe Etudiant

Méthode de la classe Etudiant



# Définition externe d'une classe

## Exemple

- Le fichier **Etudiant.cpp**

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

Etudiant::Etudiant(string n, int a) {
    nom = n;
    age = a;
}

void Etudiant::afficher() {
    cout << "Nom : " << nom << ", Âge : " << age << endl;
}
```

Appel du Fichier **Etudiant.h**

Définition du  
**constructeur** de la classe  
Etudiant

Définition de la méthode  
déclarée dans le fichier  
**Etudiant.h**



# Définition externe d'une classe

## Exemple

- Le fichier **main.cpp**

```
#include "Etudiant.h"
```

Permet d'utiliser la classe Etudiant

```
int main() {  
    Etudiant e1("Amani", 25);  
    e1.afficher();  
    return 0;  
}
```

Création d'un objet e1 de type Etudiant et initialisation de ses attributs

Exécution d'une méthode sur l'objet e1

# Notion des classes - This

---

- **En cas d'ambiguïté de nom** entre les attributs et d'autres variables dans les méthodes, on dispose du pointeur sur l'instance courante, `this`, qui nous aide à pointer les attributs de la classe.

```
void setLargeur(double largeur)
{
    this->largeur = largeur;
}
void setHauteur(double hauteur)
{
    this->hauteur = hauteur;
}
```

# Constructeurs

---

- Les constructeurs sont des méthodes particulières qui ont pour responsabilité d'initialiser les attributs de la classe.
- Le constructeur porte le même nom que la classe, n'a pas de type de retour et est invoqué automatiquement où on crée une instance.
- Une classe peut avoir plusieurs constructeurs
- Si aucun constructeur n'est spécifié le compilateur fournit un constructeur par défaut à la classe ( attention : laisse non initialisé les attributs de type de base).



## Fichier Etudiant.h

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant
{
public:
    // Les constructeurs
    Etudiant(string n, int a);
    Etudiant();

    // Les méthodes
    string getNom() const;
    int getAge() const;
    void afficher() const;

private:
    string nom;
    int age;
};

#endif
```

## Fichier Etudiant.cpp

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

// constructeur avec paramètres
Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

// constructeur par défaut
Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}

// méthodes
string Etudiant::getNom() const
{
    return this->nom;
}
```



# Constructeurs – Liste d'initialisation

Les listes d'initialisation servent pour initialiser les attributs et/ou d'appeler leurs constructeurs (s'ils sont des objets).

## Constructeurs SANS liste d'initialisation

```
#include "Etudiant.h"

Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}
```

## Constructeurs AVEC liste d'initialisation

```
Etudiant::Etudiant(string n, int a)
    : nom(n), age(a)
{
}

Etudiant::Etudiant()
    : nom(""), age(0)
{
}
```

## Avec valeur par défaut

```
Etudiant::Etudiant(string n = "", int a = 0)
    : nom(n), age(a)
{
}
```

# Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
    // suite...  
};
```

## Constructeur par défaut implicite

→ Aucun constructeur n'est défini → le compilateur génère **un constructeur par défaut implicite**, mais **les attributs ne sont pas initialisés**.

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant()  
        : nom(""), age(0)  
    {}  
    // suite...  
};
```

## Constructeur par défaut qui initialise les attributs à 0

→ Constructeur par défaut explicite qui initialise  
nom à " "  
age à 0

# Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n = "", int a = 0)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

## Constructeur avec paramètres par défaut

- Ici, **un seul constructeur**, mais il joue à la fois :
- le rôle de **constructeur paramétré**,
  - et de **constructeur par défaut** (si on appelle `Etudiant e;`  
→ utilise `n=""` et `a=0`).



# Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n, int a)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

## Pas de constructeur par défaut

- Comme il existe un constructeur paramétré **sans valeurs par défaut**,
- le compilateur **ne génère pas** de constructeur par défaut.
- Donc `Etudiant e;` n'est pas possible ❌



# Exercice

---

- On désire représenter l'objet Point caractérisé par un abscisse et un ordonné de type réel. Un point est construit de deux manières:
  - Les coordonnées sont par défaut à 0
  - Les coordonnées sont initialisés par des valeurs utilisateurs
- Un point devrait être afficher sous cette forme (1.3 , 2.9)
- Un point peut être déplacer selon l'axe des abscisses et/ou l'axe des ordonnés.
- Définir la classe point.
- Ecrire une fonction main qui crée un point A à l'origine du repère. Puis on crée un autre point B ayant des coordonnées saisis au clavier. Afficher les deux points.
- En fin, faire confondre les deux points et les afficher de nouveau.

# Exercice – Correction

```
class point
{
private:
    double abs;
    double ord;
public:
    point(double ab=0,double o=0);
    void deplacer(double, double) ;
    void afficher() const;
    double get_abs() const;
    double get_ord() const;
    void set_abs(double);
    void set_ord(double);
};
```

```
#include<iostream>
#include "point.h"

point::point(double ab, double o):abs(ab),ord(o){}
void point::deplacer(double x, double y)
{
    this->abs += x;
    this->ord += y;
}
void point::afficher() const
{
    std::cout << "(" << abs << " , " << ord << ")\n";
}
double point::get_abs() const
{
    return abs;
}
double point::get_ord() const
{
    return ord;
}
void point::set_abs(double abs)
{
    this->abs = abs;
}
void point::set_ord(double ord)
{
    this->ord = ord;
}
```



# Exercice – Correction

```
#include <iostream>
#include "point.h"

int main()
{
    double x, y;
    point A;
    std::cout << "Donner les coordonnes de B:";
    std::cin >> x;
    std::cin >> y;

    point B(x,y);
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
    std::cout << "confondre A et B ....\n";
    A.set_abs(B.get_abs());
    A.set_ord(B.get_ord());
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
}
```

# Constructeur de copie

---

**Un constructeur de copie** est appelé lors de l'instanciation d'un objet avec en argument d'appel un objet du même type. Il est aussi appelé lors du passage par valeur d'un objet en paramètre, ainsi que lors du retour d'une fonction ou méthode qui retourne un objet de ce type. Le rôle d'un constructeur par copie est de permettre l'instanciation d'un nouvel objet dans le même état qu'un objet existant (ou clone).

**Syntaxe :**

fichier Toto.h

```
class Toto {  
    ...  
    Toto( Toto & autre );  
    ...  
};
```

fichier Toto.cxx

```
#include "Toto.h"  
...  
Toto::Toto( Toto & autre )  
{ ... }
```



# Constructeur de copie

Ce genre de constructeur permet d'initialiser un objet avec un autre objet.

Exemple classe Etudiant :

---

```
Etudiant::Etudiant(Etudiant const& e)
    : nom(e.nom), age(e.age)
{
}
```

Le compilateur crée automatiquement un constructeur de copie si le développeur n'en fournit pas.

Ce constructeur de copie par défaut copie chaque attribut de l'objet, exactement comme le ferait un constructeur de copie écrit manuellement.

C'est pourquoi, dans la plupart des cas, on utilise simplement celui généré par le compilateur et on n'a pas besoin de définir un constructeur de copie nous-même.

# Destructeur d'une classe

Le **destructeur** en C++ est une fonction spéciale (méthode particulière) qui s'exécute **automatiquement** quand un objet est **détruit**.

C'est l'opposé du **constructeur**.

## Syntaxe du destructeur

Il porte le même nom que la classe, précédé d'un **tilde (~)**, et n'a aucun paramètre.

```
class Point {  
public:  
    ~Point() {  
        // Code exécuté à la destruction  
    }  
};
```

- Pas de valeur de retour
- Pas de paramètres
- Un seul destructeur par classe

# Destructeur d'une classe

Le destructeur en C++ sert à **libérer les ressources** utilisées par l'objet :

- libérer de la mémoire (delete, free)
- libérer un tableau dynamique (delete[])
- afficher un message de destruction (pour tests)

```
class Etudiant {  
private:  
    int* notes;  
public:  
    Etudiant() {  
        notes = new int[10]; // allocation  
    }  
  
    ~Etudiant() {  
        delete[] notes;      // libération  
        cout << "Etudiant supprimé" << endl;  
    }  
};
```



# Destructeur d'une classe

## A quel moment le constructeur s'exécute ?

---

### 1) Un objet local (automatique) sort de sa portée :

Quand un objet est déclaré à l'intérieur d'un bloc { }, il est détruit automatiquement à la fin de ce bloc.

```
{  
    Point A; // créé ici  
}           // A est détruit ici → destructeur appelé
```

→ Dès qu'on sort du bloc, le destructeur s'exécute.

### 2) Un objet créé avec new est détruit avec delete

Les objets créés avec new ne sont **pas détruits automatiquement**.

```
Point* p = new Point();  
delete p; // destructeur appelé ici
```

→ Il faut appeler delete → sinon fuite mémoire.



# Destructeur d'une classe

---

## 3) Un objet temporaire prend fin

Un objet temporaire est créé par le compilateur, souvent dans une expression.

Exemple :

```
Point temp = Point(3,4);
```

```
// destructeur de temp appelé juste après utilisation
```

Ou :

```
afficherPoint(Point(2,5)); // le Point(2,5) temporaire meurt après l'appel
```

➡ **Le destructeur s'exécute juste après l'utilisation du temporaire.**

## 4) Le programme se termine

À la fin du programme, tous les objets **globaux** ou **statiques** sont détruits.

```
static Point P; // créé au lancement, détruit à la fin du programme
```

➡ **Le destructeur est appelé automatiquement quand le programme quitte.**

# Surcharge d'opérateur

---

**La surcharge d'opérateur** en C++ permet de redéfinir le comportement des opérateurs intégrés (comme +, -, ==, <<) pour qu'ils fonctionnent avec des classes personnalisées, rendant le code plus intuitif, comme si les objets étaient des types primitifs.

Cela se fait en définissant une fonction spéciale nommée `operator@` (où @ est l'opérateur), soit comme fonction membre de la classe, soit comme fonction globale ou amie, en expliquant au compilateur comment l'opération doit s'appliquer aux objets. Le nombre d'arguments dépend si l'opérateur est unaire (un argument) ou binaire (deux arguments).

# Surcharge d'opérateur

**Exemple :** Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ **Surcharge de l'opérateur +:**

On va additionner les notes de deux étudiants e1 et e2

```
Etudiant operator+(const Etudiant& e1, const Etudiant& e2) {  
    return Etudiant(e1.getNom() + " & " + e2.getNom(), e1.getNote() + e2.getNote());  
}
```

**Exemple :**

```
Etudiant a("Ahmed", 12.5);  
Etudiant b("Maryam", 15.0);  
  
Etudiant c = a + b;
```

Etudiant c aura :

- nom = « Ahmed & Maryam »
- note = 27.5



# Surcharge d'opérateur

**Exemple :** Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ Surcharge de l'opérateur == :

On va comparer deux étudiants par note :

```
bool operator==(const Etudiant& e1, const Etudiant& e2) {  
    return e1.getNote() == e2.getNote();  
}
```

➤ Surcharge de l'opérateur < (Comparaison)

```
bool operator<(const Etudiant& e1, const Etudiant& e2) {  
    return e1.getNote() < e2.getNote();  
}
```



# Surcharge d'opérateur

**Exemple :** Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ **Surcharge de l'opérateur << (affichage) :**

C'est le plus utilisé : afficher un étudiant avec cout <<

```
#include <iostream>  
  
std::ostream& operator<<(std::ostream& out, const Etudiant& e) {  
    out << "Nom: " << e.getNom() << ", Note: " << e.getNote();  
    return out;  
}
```

## Exemple : Soit la classe Etudiant suivante

```
#include <iostream>
#include <string>
using namespace std;

class Etudiant {
private:
    string nom;
    float note;

public:
    Etudiant(string n = "", float no = 0) : nom(n), note(no) {}

    float getNote() const { return note; }
    string getNom() const { return nom; }
};
```

```
// Addition des notes
Etudiant operator+(const Etudiant& e1, const Etudiant& e2) {
    return Etudiant(e1.getNom() + " & " + e2.getNom(), e1.getNote() + e2.getNote());
}

// Comparaison ==
bool operator==(const Etudiant& e1, const Etudiant& e2) {
    return e1.getNote() == e2.getNote();
}

// Affichage <<
ostream& operator<<(ostream& out, const Etudiant& e) {
    out << "Nom: " << e.getNom() << ", Note: " << e.getNote();
    return out;
}

int main() {
    Etudiant a("Ahmed", 12.5);
    Etudiant b("Maryam", 15.0);

    cout << a << endl;
    cout << b << endl;

    Etudiant c = a + b;
    cout << c << endl;

    if (a == b)
        cout << "Même note" << endl;
}
```

# Surcharge Interne - Surcharge externe

## ✓ Surcharge interne (méthode membre)

**La surcharge interne** signifie que l'opérateur est défini *à l'intérieur de la classe*, comme une *méthode membre*.

Caractéristiques :

- L'opérateur est dans la classe
- Il a accès directement aux attributs privés
- Le premier opérande est toujours l'objet courant (`this`)

```
class Etudiant {  
private:  
    string nom;  
    float note;  
  
public:  
    Etudiant(string n="", float no=0) : nom(n), note(no) {}  
  
    // surcharge interne  
    Etudiant operator+(const Etudiant& e) const {  
        return Etudiant(nom + " & " + e.nom, note + e.note);  
    }  
};
```



# Surcharge Interne - Surcharge externe

✓ **Surcharge externe (fonction externe)**

**La surcharge externe** signifie que l'opérateur est défini *en dehors de la classe*, comme une *fonction indépendante*.

### Caractéristiques :

- Fonction définie **hors** de la classe
- On doit passer les objets en paramètres

```
Etudiant operator+(const Etudiant& e1, const Etudiant& e2) {
    return Etudiant(e1.getNom() + " & " + e2.getNom(),
                    e1.getNote() + e2.getNote());
}
```